Can LLMs Implement Architectural Tactics?

Anonymous Author(s)

Abstract

Software maintainability is a critical quality attribute that directly affects system evolution cost and reliability. Improving maintainability often requires architectural-level modifications demanding specialized expertise and significant manual effort. While Large Language Models (LLMs) have shown strong capabilities in code understanding and generation, their potential for systematic architecture-level software quality improvement remains largely unexplored.

This paper presents an automated pipeline that leverages an LLM to improve software maintainability through architectural tactic implementation. The pipeline operates in three stages: (1) architectural tactic selection, where the LLM chooses an appropriate tactic from a predefined catalog based on the detected architecture and expected maintainability impact; and (3) tactic implementation. The pipeline follows a Pipes and Filters architecture and operates without manual intervention.

We applied the pipeline to 57 open-source Python backend repositories collected and filtered from GitHub. Baseline maintainability was measured using the Radon static analysis tool, with the Maintainability Index as the primary metric.

This work contributes a reproducible methodology for LLM-driven architectural improvement and provides empirical evidence on the feasibility of using LLMs for architecture-level analysis and tactic reasoning.

CCS Concepts

• **Software and its engineering** → **Software maintenance tools**; *Software architectures*; • **Computing methodologies** → *Natural language processing*.

Keywords

software maintainability, architectural tactics, large language models, automated software improvement, static analysis, software architecture

ACM Reference Format:

Anonymous Author(s). 2026. Can LLMs Implement Architectural Tactics?. In *Proceedings of The 30th International Conference on Evaluation and Assessment in Software Engineering (EASE 2026).* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Modern software engineering faces a persistent maintenance challenge, with studies indicating that 60–80% of total lifecycle costs are dedicated to post-deployment modifications [2]. While code-level refactoring is well-established [4], and industrial studies confirm that developers regularly refactor to improve maintainability despite significant effort and risk [8], there is a significant gap in the automated application of higher-level design decisions—what we term the "Transformation Gap" — defined as the technical and conceptual disconnect between identifying architectural opportunities and the reliable synthesis of the corresponding code — between detecting the need for architectural change and implementing it automatically. This paper investigates the intersection of architectural tactics, quality attribute models, and the emerging capabilities of Large Language Models (LLMs) in automating structural transformations.

We address the following research questions:

- **RQ1:** To what extent can an automated LLM-based pipeline reliably identify architectural patterns and align them with appropriate tactics in diverse real-world repositories?
- **RQ2:** To what extent do LLM-implemented architectural tactics improve software maintainability as measured by the Maintainability Index?
- **RQ3:** How do architecture type and tactic choice influence improvement outcomes?

The remainder of this paper is organized as follows: Section 2 reviews background and related work; Section 3 describes the methodology; Section 4 details the implementation; Section 5 presents results and discussion; Section 6 discusses threats to validity; and Section 7 concludes.

2 Background and Related Work

2.1 Architectural Tactics and Quality Attributes

Software architecture is fundamentally defined by its elements, form, and rationale [16], or alternatively, as a collection of components and connectors in a specific configuration [5]. Within these frameworks, *architectural tactics* serve as fine-grained design decisions intended to achieve specific quality attribute responses [2]. Unlike architectural patterns (e.g., MVC, Layered), which provide broad structural templates, tactics target specific quality concerns at a finer granularity—for instance, "Reduce Coupling" targets modifiability, while "Decomposability" targets analysability and modularity. Kim et al. demonstrated that quality-driven architecture development using tactics can systematically improve targeted quality attributes when tactics are selected based on measured deficiencies [9]. Bogner et al. further showed that modifiability tactics—particularly coupling reduction—are systematically addressed through service-oriented design patterns in both SOA and microservice architectures [3].

The ISO/IEC 25010 standard [1] decomposes maintainability into five sub-characteristics: modularity, analysability, modifiability, reusability, and testability. The Maintainability Index (MI)—a composite metric combining Halstead Volume [6], Cyclomatic Complexity [14], and lines of code—has been widely adopted as a practical proxy for these sub-characteristics [15]. However, its validity

EASE 2026, Glasgow, United Kingdom

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

remains debated: Lenarduzzi et al. found less than 0.4% agreement among six static analysis tools on quality issues [10], highlighting the need for multi-metric evaluation.

Research by Rahmati suggests that while architectural patterns provide a structural foundation, their effectiveness is highly context-dependent; misapplication can lead to a 10% increase in maintenance effort [18]. Márquez et al. conducted a systematic mapping of architectural tactics in software architecture, cataloging over 40 tactics and their quality attribute impacts [12]. For the purpose of this study, we refined this catalog to a subset of 32 tactics specifically focused on enhancing maintainability and modifiability, excluding those primarily targeting performance or availability. This catalog, which maps each tactic to its primary quality attribute impact (e.g., Decomposability → Analysability, Reduced Coupling → Modifiability), forms the basis for our automated tactic selection mechanism.

2.2 LLMs in Software Refactoring

Recent multi-agent frameworks such as MANTRA have demonstrated an 82.8% success rate in producing compilable, test-passing refactored code through contextual RAG and multi-agent LLM collaboration [20]. However, empirical analysis of AI coding agents reveals a significant scale limitation: agentic refactorings are dominated by low-level, localized edits (renaming, type changes), with agents performing fewer high-level design changes than human developers [7]. This localized scope is particularly problematic for architectural tactics, which by nature require coordinated changes across multiple modules.

Research into prompt engineering indicates that while LLMs can identify up to 86.7% of refactoring opportunities, their performance in complex tactic synthesis remains limited [11]. Piao et al. explored bridging human expertise with LLM machine understanding for refactoring, finding that structured prompts with architectural context significantly improve output quality [17]. In security-related tactic implementation, Shokri and Khatchadourian found that models may achieve high syntactic correctness (95%) but fail significantly in semantic correctness (5%) due to an inability to coordinate inter-procedural changes across multiple classes [19]. Martinez et al. conducted a systematic literature review of software refactoring with LLMs, confirming that architecture-level transformations remain an open challenge [13].

2.3 Research Gaps

The synthesis of current literature identifies three key gaps that motivate this work:

- **Contextual Constraints:** Current LLM-based tools operate primarily at the method or class level, lacking system-wide awareness for architectural restructuring [7, 13]. No existing approach provides the LLM with a full repository-level architectural context.
- **Measurement Inconsistency:** There is a documented lack of agreement among static analysis tools (less than 0.4% in some cases), complicating quality validation [10]. This makes it difficult to reliably assess whether LLM-driven changes genuinely improve quality.
- **Implementation Void:** While methods exist for *detecting* the need for architectural tactics [12] and for *recovering* architectural decisions [19], no automated pipeline exists that closes the loop from detection through selection to concrete, multi-file implementation. A systematic literature review by Martinez et al. confirms that architecture-level refactoring remains an open challenge, with the vast majority of LLM-based studies addressing only code-level transformations [13].

3 Methodology

3.1 Research Design

This section describes the second stage of the architecture-aware maintainability improvement pipeline. Architecture detection, which constitutes the first stage, was evaluated separately as described in Section **??**. The ground-truth architecture labels produced by that evaluation serve as input to the tactic selection and implementation stages described here.

This study employs a quantitative pre-test/post-test experimental design. The independent variable is the application of an architectural tactic selected and implemented by an LLM, given a known repository architecture. The dependent variable is software maintainability, measured using the Maintainability Index as the primary metric. Using validated architecture labels rather than LLM-inferred classifications makes it possible to isolate the effect of tactic selection and implementation from errors introduced by the detection stage.

Each repository serves as its own control, allowing paired comparison of maintainability metrics before and after the intervention. The null hypothesis is that tactic implementation does not produce a statistically significant change in the Maintainability Index.

3.2 Pipeline Scope

This experiment covers the second half of the overall architecture-aware pipeline, consisting of the following phases:

1. **Baseline static analysis:** Computation of MI, fan-out, and docstring coverage for the original codebase;
2. **Tactic selection:** LLM-based selection of an appropriate architectural tactic from a predefined catalog, given the validated architecture label and identified structural deficiencies;
3. **Tactic implementation:** LLM-generated code modifications applying the selected tactic through iterative incremental changes with syntactic validation;
4. **Post-intervention static analysis:** Recomputation of the same metric suite on the modified codebase;
5. **Statistical analysis:** Paired comparison of before/after metrics.

Repositories that fail at any phase are recorded as null outcomes and retained in the dataset for failure rate analysis.

3.3 Dataset

The dataset for this experiment consists of the 57 manually labeled Python backend repositories from the architecture detection study. Each repository carries a validated architectural style label (script-based, layered, or modular monolith). The use of validated

labels eliminates architecture misclassification as a confounding variable and allows the evaluation to focus on tactic selection and implementation effectiveness.

Of the 57 repositories, one could not be retrieved during the experiment run due to repository deletion or access restrictions, leaving 56 unique repositories. Of these, 14 (25%) were lost during cloning and did not produce any metrics. The remaining 42 repositories produced paired before/after measurements.

3.4 Maintainability Measurement

The primary metric is the Maintainability Index (MI), computed using Radon [15] as a weighted composite of Halstead Volume [6], Cyclomatic Complexity [14], and lines of code, aligned with the ISO/IEC 25010 quality model [1]. Maintainability change is quantified as:

$$\Delta MI = MI_{after} - MI_{before} \quad (1)$$

Repositories are classified as *improved* ($\Delta MI > 0$), *stable* ($\Delta MI = 0$), or *worsened* ($\Delta MI < 0$). Structural metrics including fan-out and docstring coverage are collected as supplementary indicators.

3.5 Tactic Catalog

Tactic selection is grounded in the catalog established by Bass et al. [2] and mapped by Márquez et al. [12]. Three maintainability-oriented tactics were applied:

- **Decomposability:** splitting monolithic modules into smaller cohesive units to improve analysability;
- **Localized Modification:** encapsulating change-prone logic to reduce ripple effects;
- **Reduced Coupling:** introducing abstraction or mediators to minimize inter-module dependencies.

The LLM selects the tactic based on the validated architecture label and structural metrics, providing a rationale for the selection.

3.6 LLM-Driven Tactic Selection and Implementation

Tactic Selection. The LLM receives the repository file tree, import dependency graph, baseline static analysis metrics, and the validated architecture label. Based on identified structural deficiencies (e.g., high fan-out, monolithic files with low MI), it selects a tactic and provides a justification. If the LLM determines that no tactic is applicable, the repository is marked with outcome *(none)* and excluded from implementation.

Tactic Implementation. The implementation follows an iterative incremental strategy. The LLM generates small, self-contained code modifications one step at a time. A planner module proposes each step in structured JSON format, specifying the target file and the intended change. Each modification is validated for syntactic correctness before the next step is generated. If the planner fails to produce a valid plan after multiple retries, the repository is marked as a null outcome. The current implementation does not guarantee semantic behavior preservation; behavioral verification through automated test execution is designated as future work.

All LLM calls use the `qwen3-coder-next:cloud` model via the Ollama API with temperature 0.2 and a 256k token context window, providing the LLM with a global view of the repository to reason about cross-module dependencies [17].

4 Results and Discussion

4.1 Overall Quantitative Impact

Table 1 summarizes outcomes across 56 repositories. Of these, 14 (25.0%) failed to clone, leaving 42 with paired measurements. Of the 42 completed cases, 18 improved (42.9%), 17 remained stable (40.5%), and 7 worsened (16.7%).

Table 1: Pipeline outcome distribution (N = 56).

Outcome	N	Share
Improved ($\Delta MI > 0$)	18	42.9% of completed
Stable ($\Delta MI = 0$)	17	40.5% of completed
Worsened ($\Delta MI < 0$)	7	16.7% of completed
Failed / null	14	25.0% of all
Total	**56**	

Descriptive statistics for the 42 completed repositories are shown in Table 2.

Table 2: Maintainability Index statistics for completed repositories (N = 42).

Metric	Value
Mean ΔMI (all completed)	+1.484
Mean ΔMI (improved only)	+3.716
Mean ΔMI (worsened only)	−0.653

Compared to the initial pipeline experiment (Section **??**), where the mean ΔMI across all completed cases was +0.48 with an improvement rate of 18.2%, this experiment shows a substantially higher improvement rate (42.9%) and a larger mean gain (+1.484 vs +0.48). This difference is consistent with the use of validated architecture labels: eliminating detection errors from tactic selection produces more coherent interventions.

4.2 Notable Improvement Cases

To understand *how* the LLM improved maintainability rather than merely *how much*, this section examines the five largest improvements in detail. These cases collectively illustrate the principal mechanism through which the pipeline produces MI gains: extraction of new cohesive modules from files with low or near-zero MI scores.

kodekloudhub/webapp-color (ΔMI = +21.98, Decomposability). This repository contained a single Python file, `app.py`, with an MI of 69.27. The LLM applied the Decomposability tactic by splitting the file into three modules: a revised `app.py` (MI = 73.74), a `color.py` module encapsulating color-related logic (MI = 100), and a `config.py` module holding configuration constants (MI = 100). The repository-level average rose from 69.27 to 91.25. This case represents the idealized scenario for the pipeline: a single monolithic file with a clearly separable concern (configuration and color

mappings) whose extraction produces simple, highly maintainable modules. The magnitude of the gain is partly an artifact of the small repository size, where each new file has equal weight in the average.

AutoLab-SAI-SJTU/Paper2Rebuttal (ΔMI = +18.14, Localized Modification). This repository contained five Python files, of which `rebuttal_service.py` had an MI of 0.0, corresponding to Radon rank C, indicating an excessively large and complex file that is practically unmaintainable by static analysis criteria. The LLM identified that this file mixed multiple distinct responsibilities: PDF conversion, LLM orchestration, and arXiv API interaction. Applying the Localized Modification tactic, it extracted three dedicated modules: `pdf_converter.py` (MI = 61.6), `llm_orchestrator.py` (MI = 50.1), and `arxiv_client.py` (MI = 34.8). After extraction, `rebuttal_service.py` itself improved to MI = 55.6, as the remaining code became more focused. This case demonstrates the highest-value scenario for the pipeline: a file with MI = 0 represents a concentration of complexity that, once partially extracted, yields large absolute MI improvements. The case also illustrates that the LLM can reason about logical boundaries within a complex file, not only about file-level structure.

paulafredo/spam-api-free-fire (ΔMI = +9.40, Decomposability). This repository had four Python files with a mean MI of 67.20. The primary file, `app.py`, mixed application routing with cryptographic utility logic. The LLM applied Decomposability by extracting cryptographic operations into a dedicated `crypto.py` module, which achieved MI = 100 due to its simple, pure-function structure. The existing `app.py` improved from MI = 44.5 to 58.7 as a result of the reduced complexity. The extracted module's simplicity and clarity are a direct consequence of the LLM selecting a functionally cohesive responsibility for extraction rather than splitting code arbitrarily.

McCloudS/subgen (ΔMI = +7.93, Decomposability). The central file of this repository, `subgen.py`, had an MI of 0.0 despite the repository containing 13 Python files. The LLM identified that `subgen.py` contained environment configuration and initialization logic mixed with core processing code. By extracting environment-related code into a new `environment.py` module (MI = 66.4), the original file's complexity decreased sufficiently for Radon to assign it an MI of 100.0. This case illustrates a recurring pattern in the dataset: a single dominant file with MI = 0 can be transformed into a well-maintainable module through targeted extraction, even when the surrounding codebase is already reasonably structured.

HKUDS/FastCode (ΔMI = +2.11, Decomposability). This case is notable because it is the only example among the top improvements where the repository is large (81 Python files), demonstrating that measurable MI gains are possible at scale under specific conditions. Two files had exceptionally low MI scores: `retriever.py` (MI = 11.32) and `main.py` (MI = 4.85). The LLM created two new modules, `call_graph_builder.py` (MI = 52.6) and `retriever_coordinator.py` (MI = 100), by extracting retrieval coordination and call graph construction logic. After extraction, `retriever.py` improved to MI = 78.43 and `main.py` to MI = 76.95. While the repository-level average increased by only 2.11 points due to the dilution effect of 79 other files, the individual file-level improvements were substantial. This case suggests that the pipeline can identify and improve critical bottleneck files in large repositories, even if the aggregate MI gain is small.

Common Pattern Across Improvement Cases. Across all five cases and the broader dataset, 12 of 18 improvements involved the extraction of a new module achieving MI = 100. These extracted modules share a characteristic structure: they contain simple, self-contained functions or classes with low cyclomatic complexity and high readability. Configuration objects, pure utility functions, API clients with limited branching, and data transformation helpers consistently produce MI = 100 under the Radon formula. The LLM systematically identifies such extractable responsibilities within mixed-concern files, suggesting that it applies an implicit cohesion heuristic when decomposing modules.

A secondary improvement pattern involves files with MI = 0 (rank C). Radon assigns MI = 0 when a file's Halstead Volume and cyclomatic complexity exceed a threshold corresponding to very long, deeply nested code. In four cases, the LLM improved MI = 0 files by at least 40 points through partial extraction, without necessarily reducing the file to a simple structure. The act of removing even a fraction of the most complex logic from such files produces disproportionately large MI gains due to the non-linear relationship between file size and Halstead Volume.

4.3 Results by Repository Size

Table 3 shows results grouped by repository size, measured in number of Python files.

Table 3: Results by repository size.

Size bin	N	Median $\lvert\Delta MI\rvert$	Mean $\lvert\Delta MI\rvert$	Max $\lvert\Delta MI\rvert$
Tiny (<10 files)	11	1.04	4.83	21.98
Small (10–30 files)	9	0.76	1.50	7.93
Medium (30–80 files)	12	0.05	0.16	0.66
Large (>80 files)	10	0.00	0.28	2.11

Repository size is the strongest predictor of effect magnitude. On tiny repositories, individual file splits can produce jumps of over 20 MI points because the repository-level average is dominated by a small number of files. On large repositories (80+ files), the median change is 0.00: any single-file modification is absorbed by the averaging over dozens of files with MI in the 60–80 range.

The most striking examples demonstrate this pattern:

- *kodekloudhub/webapp-color* (ΔMI = +21.98, 1 file, Decomposability): the single file `app.py` was split into `app.py`, `color.py` (MI = 100), and `config.py` (MI = 100), raising the repository average from 69.3 to 91.2.
- *AutoLab-SAI-SJTU/Paper2Rebuttal* (ΔMI = +18.14, 5 files, Localized Modification): `rebuttal_service.py` had MI = 0.0. The LLM extracted logic into three new modules, and the original file improved to MI = 55.6.
- *McCloudS/subgen* (ΔMI = +7.93, 13 files, Decomposability): `subgen.py` improved from MI = 0.0 to MI = 100.0 after extraction.

A common pattern across 12 of the 18 improved repositories is the extraction of a new module with MI = 100 (simple data classes, configuration objects, or factory utilities), which raises the repository-level average. Four additional improvements involved rescuing files with MI = 0.0 (Radon rank C, very complex) through partial extraction.

4.4 Results by Architectural Tactic

Table 4 shows outcomes grouped by selected tactic.

Table 4: Results by architectural tactic.

Tactic	N	Impr.	Stable	Worse.	$\overline{\Delta MI}$
(none selected)	7	0	7	0	0.00
Decomposability	18	8	7	3	+2.33
Localized Modification	14	9	3	2	+1.49
Reduced Coupling	3	1	0	2	−0.16

Localized Modification shows the best risk-adjusted performance: 9 improvements, 2 worsenings, and a mean ΔMI of +1.49. This tactic encapsulates change-prone logic without requiring the creation of entirely new module boundaries, which reduces the risk of introducing below-average MI files.

Decomposability produces the highest individual gains but also the most worsenings (3 cases). This tactic is high-variance: when applied to a monolithic file with MI = 0, it can produce large improvements; when applied to an already well-maintained codebase, new extracted files may have MI below the existing repository average, pulling the mean down.

Reduced Coupling was applied to only 3 repositories, making statistical inference unreliable. However, 2 of the 3 cases worsened slightly, suggesting that introducing abstractions or mediators in small repositories can increase complexity without proportional benefit.

4.5 Results by Architectural Style

Table 5 presents results grouped by validated architecture label.

Table 5: Results by architectural style.

Architecture	N	Impr. / Worse. / Stable	$\overline{\Delta MI}$
Script-based	7	3 / 0 / 4	+5.62
Modular monolith	26	9 / 5 / 12	+0.76
Layered	9	6 / 2 / 1	+0.35

Script-based repositories continue to show the highest mean improvement (+5.62), consistent with the initial pipeline experiment. These repositories typically have few files, low MI scores due to large monolithic scripts, and clear decomposition opportunities.

Modular monolith repositories show moderate improvements on average (+0.76) but account for all five worsening cases. This is consistent with the hypothesis that already-organized codebases are sensitive to structural changes: the LLM may introduce new files or reorganize imports in ways that add complexity rather than reducing it.

Layered repositories show the highest improvement rate in absolute terms (6 of 9 improved), with a modest mean gain of +0.35. The explicit separation of concerns in layered architectures provides a more structured context for tactic application, particularly Localized Modification, which can target a specific layer without disrupting others.

4.6 Stable and Worsened Outcomes

Of the 17 stable repositories, three categories explain the null MI change:

(1) **Syntax errors in generated files (3 cases):** Radon excluded the malformed file from the average, leaving MI unchanged despite code modifications. Examples include *SoulX-FlashHead* (`inference_service.py`: unmatched }) and *zornade/visura-api* (`auth.py`: unmatched }).
(2) **Planner failure (7 cases):** The LLM failed to produce a valid modification plan after multiple retries. This occurred predominantly in very small repositories (2–8 files) where the planner could not identify meaningful structural changes.
(3) **No-op modifications:** The LLM produced changes limited to docstrings or import reorganization, which do not affect the MI formula.

The seven worsening cases share a common pattern: the LLM extracted a new module whose MI was below the existing repository average, pulling the mean downward. The maximum degradation was −1.23 MI points (*geo-seo-claude*: new `http_client.py` with MI = 39.45 vs. repository average of 50.53). Notably, all worsening cases fell within the tiny or small size bins, and no worsening exceeded −1.24 MI points.

4.7 Pipeline Failure Analysis

The 14 failed cases divide into two categories. Ten repositories failed during the cloning stage due to repository deletion, privatization, or other access issues. Five repositories (including *agentchattr*, *codex-manager*, *easytrader*, *WARP-Clash-API*, and *any-auto-register*) had a completed before-analysis but could not produce after-analysis measurements.

Technical failures during the implementation stage included:

- **Rate limiting:** Ollama API returned HTTP 429 errors during tactic selection, requiring pipeline retries.
- **Planner parse failures:** The planner produced malformed JSON or structurally invalid plans across multiple retries, triggering abort conditions.
- **Stuck loops:** One repository (*stremio-gdrive*) entered a loop in which the same file was regenerated without progress across nine consecutive iterations.
- **Invalid step proposals:** The LLM returned step descriptions with missing or malformed file paths, causing the step to be rejected.

These failure modes highlight the brittleness of multi-step LLM code generation pipelines and the need for more robust planning and validation mechanisms.

4.8 Discussion

Effect of validated architecture labels. Comparing this experiment to the initial pipeline study (Section **??**) reveals a substantial difference in improvement rate: 42.9% here versus 18.2% in the original experiment. While the datasets differ and direct comparison must be made cautiously, the improvement is consistent with the hypothesis that reducing architecture detection errors leads to more coherent tactic selection. When the LLM knows the architecture is script-based with high certainty, it can select Decomposability with confidence; when the architecture label is uncertain (as in the original pipeline), tactic selection may be less appropriate.

Repository size as the primary moderating factor. Effect magnitude depends strongly on repository size, with a weak negative correlation of −0.13 between Python file count and $|\Delta MI|$. The MI formula averages across all files; in large repositories, individual file improvements are statistically absorbed. This means the pipeline's value is concentrated in small and tiny repositories with structurally clear improvement opportunities.

Decomposability versus Localized Modification. Decomposability provides the highest potential gains but is the highest-variance tactic. It is most effective when applied to files with MI near zero (Radon rank C) and produces degradation when new extracted modules have MI below the repository average. Localized Modification is more conservative and shows a better improvement-to-worsening ratio (9:2 vs. 8:3), making it preferable when the risk of degradation must be minimized.

MI as a noisy signal for medium and large repositories. On medium repositories (30–80 files), the median $|\Delta MI|$ was 0.05, making it practically indistinguishable from measurement noise. This does not mean the LLM produced no meaningful changes—inspection of modification logs confirms code was changed in most cases—but the changes were too local to register at the repository level. Complementary metrics at the file or module level would provide more discriminating evaluation for these cases [10].

Comparison to prior work. The 25.0% failure rate is lower than the 95% semantic failure rate reported for security tactic synthesis [19], though the tasks differ in scope. The 42.9% improvement rate among completed cases compares favorably to the initial pipeline's 18.2%, and the largest gains (+21.98, +18.14) substantially exceed those in the initial study. However, the high proportion of stable outcomes (40.5%) and the absence of behavioral preservation verification limit the strength of conclusions about architectural improvement.

5 Threats to Validity

Internal validity: Architecture labels were assigned by a single annotator and used without modification. MI changes may be influenced by factors beyond the intended tactic (e.g., docstring additions, import reorganization side effects). A single LLM model was used without comparison to alternatives.

External validity: The dataset covers 56 Python backend repositories across three architectural styles. Results may not generalize to other languages, larger enterprise systems, or architectures not covered by the three-class taxonomy.

Construct validity: MI captures file-level complexity but is insensitive to module boundary quality, inter-module coupling changes, and semantic correctness. A repository may be improved structurally in ways that MI does not reflect. The absence of behavioral verification means that some "improved" cases may have altered system behavior.

Conclusion validity: With 42 completed repositories and 18 improvements, the sample is small. The size-dominated effect makes it difficult to generalize findings to medium and large repositories where the pipeline consistently produces near-zero MI changes. Results should be treated as early feasibility evidence rather than definitive performance claims.

Data Availability

The replication package for this study, including the dataset of 56 analyzed Python repositories, static analysis artifacts (Radon MI logs), is publicly available at: Zenodo.

References

[1] 2023. ISO/IEC 25010 — Systems and software engineering: Software product Quality Requirements and Evaluation (SQuaRE) — Product quality model. https://iso25000.com/en/iso-25000-standards/iso-25010. [Online; accessed 2025-12-04].

[2] Len Bass, Paul Clements, and Rick Kazman. 2021. *Software Architecture in Practice* (4th ed.). Addison-Wesley.

[3] Justus Bogner, Stefan Wagner, and Alfred Zimmermann. 2019. Using architectural modifiability tactics to examine evolution qualities of Service- and Microservice-Based Systems. *Computer Science – Research and Development* 34 (2019), 187–237. doi:10.1007/s00450-019-00402-z

[4] Martin Fowler. 2018. *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley.

[5] David Garlan and Mary Shaw. 1993. An introduction to software architecture. 1, 3 (1993).

[6] Maurice H. Halstead. 1977. *Elements of Software Science.* Elsevier.

[7] Kosei Horikawa, Hao Li, Yutaro Kashiwa, Bram Adams, Hajimu Iida, and Ahmed E. Hassan. 2025. Agentic Refactoring: An Empirical Study of AI Coding Agents. In *arXiv preprint arXiv:2511.04824.* arXiv:2511.04824

[8] Miryung Kim, Thomas Zimmermann, and Nachiappan Nagappan. 2014. An Empirical Study of Refactoring Challenges and Benefits at Microsoft. *IEEE Transactions on Software Engineering* 40, 7 (2014), 633–649. doi:10.1109/TSE.2014.2318734

[9] Suntae Kim, Dae-Kyoo Kim, Lunjin Lu, and Sooyong Park. 2009. Quality-driven architecture development using architectural tactics. 82, 8 (2009), 1211–1231.

[10] Valentina Lenarduzzi, Fabiano Pecorelli, Nyyti Saarimaki, Savanna Lujan, and Fabio Palomba. 2023. A critical comparison on six static analysis tools: Detection, agreement, and precision. 198 (2023), 111575.

[11] Bo Liu, Yanjie Jiang, Yuxia Zhang, Nan Niu, Guangjie Li, and Hui Liu. 2025. Exploring the potential of general purpose LLMs in automated software refactoring: An empirical study. *Automated Software Engineering* 32, 1 (2025). doi:10.1007/s10515-025-00500-0

[12] Gastón Márquez, Hernán Astudillo, and Rick Kazman. 2022. Architectural tactics in software architecture: A systematic mapping study. *Journal of Systems and Software* 197 (2022), 111558. doi:10.1016/j.jss.2022.111558

[13] Sofia Martinez, Luo Xu, Mariam Elnaggar, and Eman Abdullah AlOmar. 2025. Software refactoring research with large language models: A systematic literature review. *Journal of Systems and Software* (2025), 112762. doi:10.1016/j.jss.2025.112762

[14] Thomas J. McCabe. 1976. A Complexity Measure. *IEEE Transactions on Software Engineering* SE-2, 4 (1976), 308–320. doi:10.1109/TSE.1976.233837

[15] Paul Oman and Jack Hagemeister. 1992. Metrics for assessing a software system's maintainability. In *Conference on Software Maintenance.* IEEE, 337–344.

[16] Dewayne E. Perry and Alexander L. Wolf. 1992. Foundations for the study of software architecture. 17, 4 (1992), 40–52. doi:10.1145/141874.141884

[17] Yonnel Chen Kuang Piao, Jean Carlos Paul, L. Da Silva, Arghavan Moradi Dakhel, Mohammad Hamdaqa, and Foutse Khomh. 2025. Refactoring with LLMs: Bridging Human Expertise and Machine Understanding. In *arXiv preprint arXiv:2510.03914.* arXiv:2510.03914

[18] Zahed Rahmati and Mohammad Tanhaei. 2021. Ensuring software maintainability at software architecture level using architectural patterns. 2, 1 (2021), 81–102. doi:10.22060/ajmc.2021.19232.1044

[19] Seyyed Ehsan Shokri and Raffi Khatchadourian. 2024. IPSynth: Interprocedural Program Synthesis for Software Architecture Recovery and Architectural Tactics Detection. In *arXiv preprint arXiv:2403.10836*. arXiv:2403.10836

[20] Yisen Xu, Feng Lin, Jinqiu Yang, Tse-Hsun Chen, and Nikolaos Tsantalis. 2025. MANTRA: Enhancing Automated Method-Level Refactoring with Contextual RAG and Multi-Agent LLM Collaboration. In *arXiv preprint arXiv:2503.14340*. arXiv:2503.14340